

759 Final Project Proposal: Mixed-Precision Fused Attention Kernel using Tensor Cores.

Yupeng Zhang

April 22, 2026

1 Background and Motivation

The Attention mechanism is the computational bottleneck of modern AI systems (e.g., Transformers). Standard Attention implementations are strictly memory-bound; writing the intermediate $N \times N$ attention matrix to Global Memory (VRAM) severely underutilizes the computational power of the GPU. This project aims to solve this memory bottleneck by implementing a "Fused" Attention kernel. By fusing the matrix multiplications and the Softmax operation into a single kernel, intermediate results never leave Shared Memory (SRAM). Furthermore, the project will leverage the `nvcuda::wmma` API to perform mixed-precision calculations, utilizing hardware Tensor Cores to maximize compute throughput.

2 Project Scope and Constraints

To ensure the project is feasible within a four-week timeline, the scope is strictly defined:

- Precision:** Inputs (Q, K, V) and outputs are FP16. Intermediate accumulation (logits and Softmax) will be computed in FP32.
- Dimensions:** Fixed sequence length ($N = 1024$) and fixed head dimension ($d = 64$).
- Features:** Single batch, single attention head, and bidirectional attention (no causal masking).
- Hardware Target:** Development on Ada Lovelace (RTX 4070 Ti Super), with final benchmarking on Ampere (A100) and Hopper (H100) architectures.

3 Detailed 4-Week Implementation Plan

3.1 Week 1: Ground Truth and Baseline Implementation.

Task 1.1: Write a CPU C++ reference implementation of the Attention mechanism using standard math.

Task 1.2: Write a CPU C++ reference implementation of the "Online Softmax" algorithm to verify the mathematical validity of calculating Softmax in blocks.

Task 1.3: Implement a Naive (Unfused) CUDA Attention kernel using standard CUDA cores and FP32 precision. This will write the intermediate QK^T matrix to Global Memory.

Task 1.4: Verify the accuracy of the Naive GPU kernel against the CPU reference using a numerical tolerance check.

3.2 Week 2: Memory Hierarchy and The SRAM Pipeline.

Task 2.1: Design the grid and block dimensions for the Fused Kernel (e.g., configuring threads to process 128×64 output tiles).

Task 2.2: Allocate `__shared__` memory arrays for the Q , K , and V tiles.

Task 2.3: Implement the outer warp loop to collaboratively load FP16 tiles from Global Memory into Shared Memory.

Task 2.4: Implement memory padding strategies in the `__shared__` array declarations to eliminate Shared Memory bank conflicts during warp reads.

Task 2.5: Complete a Fused Attention kernel using standard CUDA cores (no Tensor Cores yet) and verify its accuracy.

3.3 Week 3: Tensor Core Integration via WMMA API.

Task 3.1: Replace standard shared-memory math with `nvcuda::wmma::fragment` declarations.

Task 3.2: Load FP16 Q and K data from Shared Memory into `matrix_a` and `matrix_b` registers.

Task 3.3: Execute `wmma::mma_sync` to compute QK^T and accumulate the results in FP32 accumulator fragments.

Task 3.4: Apply the Online Softmax rescaling math directly to the FP32 accumulator registers.

Task 3.5: Cast the FP32 Softmax results back to FP16, load V tiles into registers, and execute the final `wmma::mma_sync` to compute the output.

3.4 Week 4: Benchmarking, Roofline Analysis, and Reporting.

Task 4.1: Profile the Naive kernel and the Fused Tensor Core kernel using NVIDIA Nsight Compute (ncu).

Task 4.2: Extract Global Memory Throughput (GB/s) and Compute Throughput (TFLOPS) for both implementations across the A100 and H100 GPUs.

Task 4.3: Construct a Roofline Model chart to visually demonstrate how the Fused kernel shifts the workload from being memory-bound to compute-bound.

Task 4.4: Compile the final report detailing the math behind Online Softmax, the Shared Memory bank conflict resolution, and the hardware profiling results.

4 Expected Deliverables

- C++/CUDA source code for the CPU Reference, Naive Kernel, and Fused Tensor Core Kernel.
- A comprehensive final report featuring the Roofline Analysis and performance scaling graphs.

5 About

The proposal plan is generated by gemini after discussing my goal and practical limitations.